



深圳市晟峰光电科技有限公司

SFGD 光机控制 SDK

PROJECT SOLUTIONS

SunFun-SF4710-SDK V1.4

2025-09-15 更新



文档标题：	SFGD 光机控制动态库 4710SDK 文档
文档版本：	v1.4
发布日期：	2025-05-16
最后更新：	2025-09-15
作者/负责人：	叶进发
审核人：	王哲
适用动态库版本：	v1.4
适用 DLP 平台：	DLP4710LC、DLPC3479
适用 DLP 固件版本：	v8.3.0 或以上
适用 MCU 固件版本：	v2.2 或以上
支持的软件平台	Windows 10 及 11 版本、Ubuntu18.04 及以上版本、Qt5 及以上版本

更新记录：

日期	版本	更新内容
2025-05-16	v1.0	创建，初始版本
2025-06-04	V1.1	添加 Linux 平台注意事项
2025-08-18	V1.2	添加 dlphandle.lib 库
2025-09-10	V1.3	添加生成 pattern data 接口（蓝色高亮部分）
2025-09-15	V1.4	解决调试信息未配置出现段错误问题

一、 产品简介

SF47XX 系列是深圳市晟峰光电科技有限公司推出的工业级高精度结构光光机，专为 3D 扫描、3D 自动光学检测 (AOI) 及机器视觉应用设计。本产品基于德州仪器(TI) DLP4710 数字微镜器件(DMD)显示技术开发，具有高分辨率、高稳定性和工业级可靠性等特点。

二、 SDK 功能概述

本光机控制动态库 SDK 提供完整的设备控制接口，主要功能包括但不限于：

- 设备状态管理：实时读取光机工作状态、温度等参数
- 投影控制：精确控制结构光投影模式及参数
- 时序管理：灵活配置投影时序和触发信号
- 光源调节：动态调整 LED 电流及配置开机亮度
- 固件维护：支持固件备份、恢复及升级操作
- 系统配置：设备参数配置与持久化存储

三、 兼容性说明

- 支持平台：Windows 10(64 位)/ Windows 11(64 位)，Qt5 及以上版本
- 连接方式：USB 转串口或直接 RS232 串口连接，波特率 115200
- 目标设备：SF47XX 系列光机及兼容型号

四、 文档用途

本文档面向集成开发人员，详细说明动态库 API 的使用方法、参数规范及开发注意事项，帮助开发者快速实现光机设备的集成与控制。

如果对光机功能还不是很了解，请先研究使用 **SF_GUI_tool** 及查阅 **SF_GUI Tool 使用说明**。

五、 目录文件说明：

SF_dlp:	库文件目录
test_dlpc_v01:	测试程序的所有代码
test_dlpc_app:	可执行的测试程序

注意：

- 1- 在执行 windeployqt test_dlpc_v01.exe 打包程序时，需要将 SF_dlp\lib_win 目录下的 dlphandle.dll 拷贝到与 test_dlpc_v01.exe 同级目录下
- 2- 在进行 Linux 开发时，需将 SF_dlp\lib_linux 目录下的 lib_linux.tar.xz 文件复制到 Linux 环境下工程目录，再使用 tar -Jxf lib_linux.tar.xz 进行解压（解压后若需要移动文件夹的位置，可通过 cp -rdvf 命令进行复制，保持链接文件的依赖性，请勿在 windows 下解压后再传入 Linux 中），解压后如下图

```

drwxrwxr-x 5 zhangya zhangya 4.0K 5月 21 10:44 ../
lrwxrwxrwx 1 zhangya zhangya 22 5月 21 10:42 libdlphandle.so -> libdlphandle.so.1.0.0*
lrwxrwxrwx 1 zhangya zhangya 22 5月 21 10:42 libdlphandle.so.1 -> libdlphandle.so.1.0.0*
lrwxrwxrwx 1 zhangya zhangya 22 5月 21 10:42 libdlphandle.so.1.0 -> libdlphandle.so.1.0.0*
-rwxrwxr-x 1 zhangya zhangya 3.8M 5月 21 10:42 libdlphandle.so.1.0.0*
lrwxrwxrwx 1 zhangya zhangya 21 5月 21 10:42 libdlphandle.so -> libdlphandle.so.1.0.0*
lrwxrwxrwx 1 zhangya zhangya 21 5月 21 10:42 libdlphandle.so.1 -> libdlphandle.so.1.0.0*
lrwxrwxrwx 1 zhangya zhangya 21 5月 21 10:42 libdlphandle.so.1.0 -> libdlphandle.so.1.0.0*
-rwxrwxr-x 1 zhangya zhangya 422K 5月 21 10:42 libdlphandle.so.1.0.0*

```

六、 接口清单

序号	接口名称	API 名称	对应串口命令码
1	动态库配置初始化	SF_initLibrary	
2	获取动态库版本	SF_getLibraryVersion	
3	配置串口信息，波特率： 115200、无校验	SF_openUart	
4	关闭串口	SF_closeUart	
5	检测设备是否就绪	SF_deviceCheckReady	
6	输出配置错误信号	SF_uartErrorSign	
7	串口数据包信号	SF_uartPackSign	
8	串口命令数据信号	SF_uartCmdDataSign	
9	处理串口接收上报信号	SF_handleRxReportSign	
10	处理串口接收错误信号	SF_handleRxErrorSign	
11	输出传输状态	SF_transmitStatusSign	
12	输出传输进度	SF_transmitProgressSign	
命令区			
13	配置 DLP 状态	SF_setDlpPower	01
14	软触发	SF_softTrigger	02
15	设置蓝灯亮度	SF_setBlueBrigh	03
16	读取蓝灯亮度	SF_getBlueBrigh	04
17	复位 MCU	SF_resetMCU	30
18	获取版本号	SF_getVersion	31
19	恢复出厂设置	SF_resetFactory	32
20	获取软件编译时间	SF_getBuildTime	33
21	更新触发时间	SF_updateTriggerTime	34
22	控制 MCU 接口	SF_ctlHardware	35
23	设置用户数据	SF_setUserData	36
24	获取用户数据	SF_getUserData	37
25	获取设备 ID	SF_getDeviceID	39
26	获取告警温度	SF_getAlarmTemp	3B
27	获取运行状态	SF_getRunStatus	3C
28	设置启动 IIC 命令	SF_setStartIIC	3D
29	获取启动 IIC 命令	SF_getStartIIC	3E
30	删除启动 IIC 命令	SF_delStartIIC	3F
31	清除启动 IIC 命令	SF_clrStartIIC	40

32	设置启动 LED 电流命令	SF_setStartCurrent	41
33	获取启动 LED 电流命令	SF_getStartCurrent	42
34	清除启动 LED 电流命令	SF_delStartCurrent	43
35	设置输入触发参数	SF_setTriggerInPara	44
36	获取输入触发参数	SF_getTriggerInPara	45
37	设置触发结束参数	SF_setTriggerEndPara	46
38	获取触发结束参数	SF_getTriggerEndPara	47
39	触发输出	SF_outPulseSignal	48
40	获取触发输入计数	SF_getTriggerCount	49
41	清除触发输入计数	SF_clrTriggerCount	4A
42	获取上电配置的 pattern time 文件名，即开机时序索引	SF_getStartupPT	0B
43	设置测试图案	SF_setTestPattern	2A XX
44	设置启动画面	SF_setSplashImages	2A XX
45	获取操作模式	SF_getOperationMode	2B XX
46	设置操作模式	SF_setOperationMode	2A XX
47	切换到外部视频	SF_switchExternalVideo	2A XX
48	设置色温	SF_setColorTemp	2A XX
49	获取色温	SF_getColorTemp	2B XX
50	设置图像镜像	SF_setImageMirror	2A XX
51	获取图像镜像	SF_getImageMirror	2B XX
52	设置 RGB 灯使能	SF_setRGBEnable	2A XX
53	获取 RGB 灯使能	SF_getRGBEnable	2B XX
54	设置 RGB 灯电流	SF_setRGBCurrent	2A XX
55	获取 RGB 灯电流	SF_getRGBCurrent	2B XX
56	获取 RGB 灯最大电流	SF_getRGBMaxCurrent	2B XX
57	设置触发和模式就绪信号	SF_setTriggerAndPatternSign	2A XX
58	获取触发和模式就绪信号	SF_getTriggerAndPatternSign	2B XX
59	显示和控制图案	SF_setPatternControl	2A XX
60	获取序列错误码	SF_getSequenceError	2B XX
61	读取有效的曝光时间	SF_readValidateExposureTime	2B XX
62	设置时序表	SF_writePatternOrderTableEntry	2A XX
63	从 flash 中加载时序表	SF_loadPatternOrderTableEntryFromFlash	2A XX
64	写入串口直通 DLP 数据	SF_writeDataToDLP	2A
65	读取串口直通 DLP 数据	SF_readDataFromDLP	2B
66	从 MCU Flash 中加载 pattern time 到 DLP 中	SF_loadPTtoDLPfromFlash	05
67	通过串口加载 pattern time 到 DLP 中	SF_updatePTtoDLPfromAPP	06
68	从 DLP 中读取 pattern time 数据到串口	SF_getPTtoAPPfromDLP	07
69	生成 pattern data 数据	SF_generatePatternData	
70	通过串口加载 pattern bin 到	SF_updatePBtoDLPfromAPP	09

	DLP 中		
71	从 DLP 中读取 pattern bin 数据到串口	SF_getPBtoAPPfromDLP	0A
72	通过串口加载 pattern time 到 MCU flash 中	SF_updatePTtoFlashFromAPP	B0
73	通过串口读取 MCU flash 中指定的 pattern time	SF_getPTtoAPPFromFlash	BB
74	从 MCU flash 中获取所有的文件名	SF_getFileNameFromFlash	BD
75	删除 MCU flash 中指定文件	SF_delFileFromFlash	BE
76	清除 MCU flash 中所有文件	SF_clrAllFileFromFlash	BF
77	通过串口升级 DLP 程序	SF_updateDLPPProgram	B9
78	从 DLP 中获取主 DLP flash 的所有程序	SF_getDLPPProgramFromDLP	0C
79	从 MCU flash 中升级 DLP 程序	SF_updateDLPFromFlash	BA
80	MCU 程序升级	SF_updateMCUProgram	B8

七、 接口 API 功能

1、动态库配置初始化

函数原型：int SF_initLibrary(void);

函数说明：

2、获取动态库版本号

函数原型：int SF_getLibraryVersion(int & mainVer, int & subVer);

函数说明：mainVer 主版本号，subVer 次版本号

3、配置串口信息

函数原型：int SF_openUart(const QString &com)

函数说明：默认串口配置为 115200，无校验

4、关闭串口

函数原型：int SF_closeUart(void);

函数说明：关闭当前串口

5、检测设备是否就绪

函数原型：int SF_deviceCheckReady(void)

函数说明：DE_UART_NOT_OPEN：检测当串口是否打开

DE_UART_BUSY：串口是否忙，进行 ymodem 传输需要时间

DE_DEVICE_BUSY：MCU 在忙中

6、输出配置错误信号

函数原型: emit SF_uartErrorSign(int errCode)

函数说明: 串口错误信号,参考 enum dlpError_e 错误码, 这里主要是串口硬件断开、串口打开失败、数据发送超时等错误

```
enum dlpError_e{
    DE_SUCCESS                      = 0,
    DE_FAIL,
    DE_OPEN_FILE_ERR,               //打开文件失败
    DE_PARAMETER_ERR,              //参数错误
    DE_DEVICE_BUSY,                //设备忙

    /* start uart */
    DE_UART_ERR                     = 50,
    DE_UART_OPEN_FAIL,             //串口打开失败
    DE_UART_TIMEOUT,               //串口通信超时
    DE_UART_DISCONNECT,            //串口被强制断开连接
    DE_UART_NOT_OPEN,              //串口未打开
    DE_UART_BUSY,                  //串口忙
    /* end uart */

    /* start cmd */
    DE_CMD_ERR                      = 100,
    DE_CMD_DATA_LEN_ERR,           //命令数据长度错误
    DE_CMD_TIMEOUT,                //命令超时
    DE_CMD_RX_DATA,                //命令数据错误
    DE_CMD_RX_DATA_LEN,           //命令数据长度错误
    /* end cmd */

    /*start pt and pb data*/
    DE_PT_AND_PB_ERR                = 150,
    DE_PACK_DATA_ERR,              //pattern bin 数据包有问题, 包首没有添加 PATN 字符
    DE_FIRST_TRANSMIT,             //pattern time 或者 pattern bin 首包传输过程中出现错误
    DE_TRANSMITING,                //pattern time 或者 pattern bin 传输过程中出现错误
    DE_TIME_ERR,                   //pattern time 显示时间太小
    DE_PRE_TIME_ERR,               //pattern time 前曝光时间太小
    DE_POST_TIME_ERR,              //pattern time 后曝光时间太小
    DE_IMAGE_OVER_LIMIT,           //pattern bin 图案数量超过限制
    DE_PT_INDEX_ERR,               //pattern time 中索引超过了 pattern sets
    DE_PT_NUM_ERR,                 //pattern time 中的选择索引超过了最大图片数量
    DE_IMAGE_SIZE_ERR,             //pattern bin 中选择图像大小错误
    /*end pt and pb data*/

    /*start update file */
```

```

DE_UPDATE_ERR                = 200,
DE_UPDATE_FILE_ERR,          //升级文件有误
/*end update file*/
};

```

7、串口数据包信号

函数原型: void SF_uartPackSign(dataDire_e dir, const QByteArray & pack);

函数说明: 串口发送或接收的原始数据包

```

enum dataDire_e{
    DD_RECEIVE,    //接收数据
    DD_TRANSMIT    //发送数据
}

```

8、串口命令数据信号

函数原型: void SF_uartCmdDataSign(dataDire_e dir, quint8 cmd, const QByteArray & data);

函数说明:

dataDire_e - 参考上面定义
cmd - 定义的命令
data - 发送或接收解析后的数据

9、处理串口接收上报信号

函数原型: void SF_handleRxReportSign(quint8 cmd, const QByteArray & data);

函数说明: 处理串口上报的命令与数据

10、处理串口接收错误信号

函数原型: void SF_handleRxErrorSign(quint8 cmd, int errCode);

函数说明: 这里主要是 MCU 端接收到的命令后处理过程中出现的问题上报给 PC 端进行提示

cmd - 当前传输的命令码
errCode - MCU 端返回的错误码, 参考 enum cmdErrCode

```

enum cmdErrCode{
    CEC_OK = 0x00,
    CEC_FAIL,
    CEC_CMD_CHECK,          //命令校验错误, 这里主要是指命令的校验和有误
    CEC_CMD_INVALID,       //命令无校,没有查打到该命令
    CEC_CMD_DATA_LEN,      //命令数据错误,超出范围
    CEC_CMD_DATA,          //数据错误
    CEC_CMD_PASSWORD,      //密码错误
    CEC_CMD_DATA_HANDLE,   //数据处理错误
    CEC_CMD_DLP_PWR_OFF,   //DLP 电源关闭
    CEC_CMD_STORAGE_IS_FULL, //存储空间已满
    CEC_CMD_NOT_FILE,      //没有该文件
    CEC_CMD_OPERATE,       //该命令操作失败
    CEC_CMD_DATA_EMPTY,    //读取命令中数据为空
    CEC_CMD_CRC,           //校验 CRC 错误, 这里主要是指传输大文件时 CRC 校验错误
}

```



```
CEC_CMD_12V_OFF          //DLP 的 12V 关闭
};
```

11、 输出传输状态

函数原型: void SF_transmitStatusSign(transmitStatus_e status);

函数说明:

```
enum transmitStatus_e{
    TS_NONE = 0,
    TS_START,          //传输开始
    TS_FINISH,         //传输完成
    TS_ERROR,          //传输错误
    TS_EXIT            //传输退出
};
```

12、 输出传输进度

函数原型: void SF_transmitProgressSign(int progress);

函数说明: 传输的进度 【0-100】

串口类接口

13、 配置 DLP 状态(重启、启动、关闭)

函数原型: int SF_setDlpPower(const dlpStatus_e status)

函数说明:

```
enum dlpStatus_e{
    DS_REBOOT = 0x01,  //重启, 发送重启命令后关闭 DLP 设备 1.5 秒后再启动 DLP 设备
    DS_START,          //启动
    DS_STOP            //关闭
}
```

14、 软触发

函数原型: int SF_softTrigger(const quint8 triCnt, const quint16 interval)

函数说明:

```
triCnt = 0: 中止软触发
triCnt = 1: 单次软触发
triCnt = 【2-254】:多次软触发, interval: 必须大于 15ms
triCnt = 255 : 无限次软触发, interval: 必须大于 15ms
```

15、 设置蓝灯亮度

函数原型: int SF_setBlueBrigh(const quint16 bright)

函数说明: 设置蓝灯亮度, 设置范围 【0-65535】

16、 读取蓝灯亮度

函数原型: int SF_getBlueBrigh(quint16 & bright)

函数说明: 读取蓝灯亮度

17、 复位 MCU

函数原型: int SF_resetMCU(void)

函数说明: 复位 MCU 命令, 500ms 保存 MCU 数据, 700ms 后复位 MCU

18、 获取版本号

函数原型: int SF_getVersion(Qstring &outVersion)

函数说明: 获取 MCU 程序的版本号

19、 恢复出厂设置

函数原型: int SF_resetFactory(void)

函数说明:

1. 将系统参数配置成默认参数
2. 清除 MCU flash 中保存的所有 pattern time 和 pattern bin 文件
3. 处理完后 700ms 重启 MCU 程序

20、 获取软件编译时间

函数原型: int SF_getBuildTime(Qstring & outBuildTime)

函数说明: 获取 MCU 程序的编译时间, 格式: Oct 11 2024 表示 2024 年 10 月 11 日

21、 更新触发时间

函数原型: int SF_updateTriggerTime(quint32 & outTime)

函数说明: 更新触发时间, 并将触发时间返回

22、 控制 MCU 接口

函数原型: int SF_ctlHardware(const ctlStatus_e status)

函数说明:

```
enum ctlStatus_e{
    CS_OPEN_12V = 0x01,    //打开 12V 电源
    CS_CLOSE_12V,          //关闭 12V 电源
    CS_SPI_TO_MCU,          //SPI 连接到 MCU
    CS_SPI_TO_USB,          //SPI 连接到 USB
    CS_OPEN_PARKZ_PIN,      //打开 PARKZ 引脚
    CS_CLOSE_PARKZ_PIN,     //关闭 PARKZ 引脚
    CS_OPEN_DLPCM_PIN,      //打开 DLPCM 引脚
    CS_CLOSE_DLPCM_PIN     //关闭 DLPCM 引脚
}
```

23、 设置用户数据

函数原型: int SF_setUserData(const QByteArray & data)

函数说明: 设置用户数据, 用户数据最大长度: DEF_USER_DATA_MAX_NUM (23 字节)

24、 获取用户用据

函数原型: int SF_getUserData(QbyteArray & outData)

函数说明: 读取用户数据

25、 获取设备 ID

函数原型: int SF_getDeviceID(QByteArray & outDeviceID)

函数说明: 读取设备 ID 数据

26、 获取告警温度

函数原型: int SF_getAlarmTemp(quint8& mcuTemp, quint8& dlpTemp)

函数说明: 读取告警温度值

27、 获取运行状态

函数原型: int SF_getRunStatus(runStatus_t & outStatus)

函数说明: 获取运行状态

```
struct runStatus_t{
    quint8 dlpStatus;           //DLP 状态 false:关机, true:开机
    quint8 mcuTemp;             //MCU 温度
    quint8 dlpTemp;             //DLP 温度
    bool mcuTempAlarm;          //MCU 告警温度
    bool ledTempAlarm;          //DLP 告警温度
    bool power12Vstate;         //12V 电源状态 false:关闭 12V 电源,true:打开 12V 电源
    bool spiPinState;           //SPI 引脚状态 false:USB spi, true:MCU spi
    bool parkzPinState;         //parkz 引脚状态
    bool dlpcmPinState;         //dlpcm 引脚状态
}
```

28、 设置启动 IIC 命令

函数原型: int SF_setStartIIC(const int index, const quint8 cmd, const QByteArray & data)

函数说明: 设置启动 IIC 命令

index:取值范围[1-9], 宏定义 DEF_START_IIC_MIN_INDEX 和 DEF_START_IIC_MAX_INDEX

29、 获取启动 IIC 命令

函数原型: int SF_getStartIIC(const int index, quint8 & outCmd, QByteArray & outData)

函数说明: 获取启动 IIC 命令

index:取值范围[1-9], 宏定义 DEF_START_IIC_MIN_INDEX 和 DEF_START_IIC_MAX_INDEX

30、 删除启动 IIC 命令

函数原型: int SF_delStartIIC(const int index)

函数说明: 删除启动 IIC 命令

index:取值范围[1-9], 宏定义 DEF_START_IIC_MIN_INDEX 和 DEF_START_IIC_MAX_INDEX

31、 清除启动 IIC 命令

函数原型: int SF_clrStartIIC(void)

函数说明: 清除启动 IIC 命令

32、 设置启动 LED 电流命令

函数原型: int SF_setStartCurrent (const rgbCurrent_t & current)

函数说明: 设置 MCU 启动时向 DLP 配置的 LED 电流参数

```
struct rgbCurrent_t{
    quint16 redCurrent;      //红灯电流
    quint16 greenCurrent;    //绿灯电流
    quint16 blueCurrent;     //蓝灯电流
}
```

33、 获取启动 LED 电流命令

函数原型: int SF_getStartCurrent(rgbCurrent_t & outCurrent)

函数说明: 读取 MCU flash 中保存的启动 LED 电流参数

34、 清除启动 LED 电流命令

函数原型: int SF_delStartCurrent(void)

函数说明: 清除 MCU flash 中保存的启动 LED 电流配置参数

35、 设置输入触发参数

函数原型: int SF_setTriggerInPara(const triggerIn_t & trIn)

函数说明:

```
struct triggerIn_t{
    bool enable;      //使能, 0: 关闭输入触发, 1: 打开输入触发
    bool polar;       //极性, 0: 低电平有效, 1: 高电平有效
    int in1HoldTime;  //输入 1 保持时间[1-100]ms
    int in2HoldTime;  //输入 2 保持时间[1-100]ms
}
```

36、 获取输入触发参数

函数原型: int SF_getTriggerInPara(triggerIn_t & outTriIn)

函数说明: 获取输入触发参数 triggerIn_t 参考上面

37、 设置触发结束参数

函数原型: int SF_setTriggerEndPara(const triggerEnd_t & triEnd)

函数说明:

```
struct triggerEnd_t{
    bool enable;      //使能, 0: 关闭输出触发, 1: 打开输出触发
    bool polar;       //极性, 0: 低电平有效, 1: 高电平有效
    int outDelay;     //输出延时, 数据范围[-999, 999]ms
    int outHoldTime;  //输出极性保持时间
}
```

38、 获取触发结束参数

函数原型: int SF_getTriggerEndPara(triggerEnd_t & outTriEnd)

函数说明: 获取触发结束参数 triggerEnd_t 参考上面

39、 触发输出

函数原型: int SF_outPulseSignal(const int time)

函数说明: 触发引脚 (PA3) 输出高电平, time 表示输出高电平保持时间 (单位: ms)

40、 获取触发输入计数

函数原型: int SF_getTriggerCount(quint32 & outCount)

函数说明: 产生一次硬触发输入时加一

41、 清除触发输入计数

函数原型: int SF_clrTriggerCount(void)

函数说明: 清除硬触发输入计数

42、 获取当前上电配置 pattern time 文件名

函数原型: int SF_getStartupPT(QString & outFileName)

函数说明: 返回以.pt 后缀的文件名

43、 设置测试图案

函数原型: int SF_setTestPattern(const testPattern_e pattern)

函数说明:

```
enum testPattern_e{
    TP_CHECKERBOARD,          //棋盘
    TP_WHITE_COLOR,           //白图
    TP_BLACK_COLOR,           //黑图
    TP_GRID,                  //网格
    TP_COLOR_RAMP              //色阶
}
```

44、 设置 splash 图像

函数原型: int SF_setSplashImages(const quint8 splash)

函数说明: 选择要显示的启动画面的索引, 数据范围<0-255>

45、 获取操作模式

函数原型: int SF_getOperationMode(operatMode_e & outMode, QString & outModeStr)

函数说明:

```
enum operatMode_e{
    OM_EXTERNAL_VIDEO_PORT = 0,      // 0x0-External Video Port
    OM_TEST_PATTERN_GENERATOR,       // 0x1-Test Pattern Generator
    OM_SPLASH_SCREEN,               // 0x2-Splash Screen
    OM_SENS_EXTERNAL_PATTERN,        // 0x3-External Pattern Streaming
    OM_SENS_INTERNAL_PATTERN,        // 0x4-Internal Pattern Streaming
    OM_SENS_SPLASH_PATTERN,          // 0x5-Splash Pattern Streaming
    OM_STANDBY=0xFF                  // 0xFF-Standby
}
```

46、 设置操作模式

函数原型: int SF_setOperationMode(const operatMode_e mode)

函数说明: 设置 DLP 的操作模式, 按照 enum operatMode_e 类型进行配置

47、 切换到外部视频

函数原型: int SF_switchExternalVideo(void)

函数说明: 切换到外部视频输出端口

48、 设置色温

函数原型: int SF_setColorTemp(const quint8 colorTemp)

函数说明: 设置色温

49、 获取色温

函数原型: int SF_getColorTemp(quint8 & outColorTemp)

函数说明: 获取色温

50、 设置图像镜像

函数原型: int SF_setImageMirror(const imageMirror_e mirror)

函数说明:

```
enum imageMirror_e{
    IF_MIRROR_NONE,           //无镜像
    IF_MIRROR_VERTICAL,       //长轴镜像
    IF_MIRROR_HORIZONTAL,     //短轴镜像
    IF_MIRROR_ROTATE          //长轴和短轴镜像
}
```

51、 获取图像镜像

函数原型: int SF_getImageMirror(imageMirror_e & outMirror)

函数说明: 获取图像镜像, imageMirror_e 参考上面

52、 设置 RGB 灯使能

函数原型: int SF_setRGBEnable(bool red, bool green, bool blue)

函数说明: 设置 RGB 灯使能

53、 获取 RGB 灯使能

函数原型: int SF_getRGBEnable(bool & red, bool & green, bool & blue)

函数说明: 获取 RGB 灯使能状态

54、 设置 RGB 灯电流

函数原型: int SF_setRGBCurrent(quint16 red, quint16 green, quint16 blue)

函数说明: 设置 RGB 灯的电流大小

55、 获取 RGB 灯电流

函数原型: int SF_getRGBCurrent(quint16 & red, quint16 & green, quint16 & blue)

函数说明: 获取 RGB 灯的电流大小

56、 获取 RGB 灯最大电流

函数原型: int SF_getRGBMaxCurrent(quint16 & maxRed, quint16 & maxGreen, quint16 & maxBlue)

函数说明: 获取 RGB 灯的最大电流值

57、 设置触发和模式就绪信号

函数原型: int SF_setTriggerAndPatternSign(const readySignal_t & readySignal)

函数说明:

```
struct readySignal_t{
    bool triggerOut1Enable;    //DLP 触发输出 1 使能
    bool triggerOut1Invert;    //DLP 触发输出 1 极性
    int triggerOut1Delay;      //DLP 触发输出 1 延时
    bool triggerOut2Enable;    //DLP 触发输出 2 使能
    bool triggerOut2Invert;    //DLP 触发输出 2 极性
    int triggerOut2Delay;      //DLP 触发输出 2 延时
    bool triggerInEnable;      //DLP 触发输入使能
    bool triggerInPolarity;    //DLP 触发输入极性
    bool patternReadyEnable;   //DLP 图案就绪使能
    bool patternReadyPolarity; //DLP 图案就绪极性
}
```

58、 获取触发和模式就绪信号

函数原型: int SF_getTriggerAndPatternSign(readySignal_t & outReadySignal)

函数说明: readySignal_t 参数说明参考上面

59、 显示和控制图案

函数原型: int SF_setPatternControl(patternCtl_e pattern, quint8 repeatCount)

函数说明:

```
enum patternCtl_e{
    PC_START = 0,    //开始
    PC_STOP,         //停止
    PC_PAUSE,        //暂停
    PC_STEP,         //单步
    PC_RESUME,       //继续
    PC_RESET         //重新开始
}
```

60、 获取序列错误码

函数原型: int SF_getSequenceError(bool & error)

函数说明: 序列码错误则返回 true, 否则返回 false

61、 读取有效的曝光时间

函数原型: int SF_readValidateExposureTime(patternMode_e PatternMode, sequenceType_e BitDepth, uint32_t ExposureTime, validateExposureTime_t &ValidateExposureTime)

函数说明:

```

struct validateExposureTime_t
{
    exposureTimeSupported_e ExposureTimeSupported;
    zeroDarkTimeSupported_e ZeroDarkTimeSupported;
    uint32_t MinimumExposureTime;
    uint32_t PreExposureDarkTime;
    uint32_t PostExposureDarkTime;
};

```

62、 设置时序表

函数原型：int SF_writePatternOrderTableEntry(writeControl_e WriteControl, patternOrderTableEntry_t &PatternOrderTableEntry)

函数说明：设置时序表到 DLP 中

63、 从 flash 中加载时序表

函数原型：int SF_loadPatternOrderTableEntryFromFlash(void)

函数说明：从 flash 中重新加载时序表

64、 写入串口直通 DLP 数据

函数原型：int SF_writeDataToDLP(const quint8 cmd, const QByteArray & writeData)

函数说明：串口直通 IIC 协议，用于配置 DLP

Cmd: IIC 协议中配置 DLP 的命令

writeData: IIC 协议中配置 DLP 数据

65、 读取串口直通 DLP 数据

函数原型：int SF_readDataFromDLP(const quint8 cmd, const QByteArray & writeData, const quint8 readLen, QByteArray & outReadData)

函数说明：串口直通 IIC 协议，用于读取 DLP 参数

Cmd: IIC 协议中读取 DLP 数据的命令

writeData: IIC 协议中需要先配置参数，再进行读取

readLen: IIC 协议中需要读取数据的长度

outReadData: 输出读取出来的参数

例如：0x99 命令，读取索引 0x01 区域数据

Cmd = 0x99 writeData = QByteArray() << 0x01 readLen = 23 即可

66、 从 MCU Flash 中加载 pattern time 到 DLP 中

函数原型：int SF_loadPTtoDLPfromFlash(bool save, QString & fileName)

函数说明：从 MCU Flash 中加载 pattern time 文件到 DLP 中

save: 0 表示不保存，重启后不会自动加载该 pattern time 文件

1 表示保存，设备重启后会自动加载该 pattern time 文件

fileName: 保存在 MCU Flash 文件名，必须以.pt 后缀，可通过 int getFileNameFromFlash(int & flashUsed, QStringList & fileName) 去获取 MCU Flash 中有哪些.pt 文件名

67、 通过串口加载 pattern time 到 DLP 中

函数原型: int SF_updatePTtoDLPfromAPP(const QByteArray & ptData, quint8 timeout = DEF_TIMEOUT)

函数说明: 通过串口将 pattern time 数据升级到 dlp 设备中

68、从 DLP 中读取 pattern time 数据到串口

函数原型: int SF_getPTtoAPPfromDLP(QByteArray & outPtData, quint8 timeout = DEF_TIMEOUT)

函数说明: 通过串口从 DLP 读取 pattern time 数据

69、生成 pattern data 数据

函数原型: int SF_generatePatternData(const std::vector<INT_PAT_PatternSet_t> & PatternSets, const std::vector<INT_PAT_PatternOrderTableEntry_t> & PatternOrderTableEntries, bool eastWestFlip, bool longAxisFlip, QByteArray & outPatternData)

函数说明: 通过 PatternSets 和 PatternOrderTableEntries 参数, 生成 pattern data 数据到 outPatternData

```
struct INT_PAT_PatternSet_t
{
    INT_PAT_BitDepth_e    BitDepth;
    INT_PAT_Direction_e   Direction;
    uint32_t              PatternCount;
    INT_PAT_PatternData_t* PatternArray;
};

struct INT_PAT_PatternOrderTableEntry_t
{
    uint8_t               PatternSetIndex;
    uint8_t               NumDisplayPatterns;
    INT_PAT_IlluminationSelect_e   IlluminationSelect;
    bool                  InvertPatterns;
    uint32_t              IlluminationTimeInMicroseconds;
    uint32_t              PreIlluminationDarkTimeInMicroseconds;
    uint32_t              PostIlluminationDarkTimeInMicroseconds;
};
```

70、通过串口加载 pattern bin 到 DLP 中

函数原型: int SF_updatePBtoDLPfromAPP(const QByteArray & pbData, quint8 timeout = DEF_TIMEOUT)

函数说明: 通过串口升级 pattern bin 到 DLP 中

71、从 DLP 中读取 pattern bin 数据

函数原型: int SF_getPBtoAPPfromDLP(QByteArray & outPbData, quint8 timeout = DEF_TIMEOUT)

函数说明: 从 DLP 中读取 pattern bin 数据

outPbData: 获取的 pattern bin 数据

72、加载 pattern time 到 MCU flash 中

函数原型: int SF_updatePTtoFlashFromAPP (const QString & flashName, const QByteArray & ptData, quint8 timeout = DEF_TIMEOUT)

函数说明: 加载 pattern time 数据到 MCU flash 中

flashName: MCU flash 中存储的 pattern time 数据文件名, 以.pt 结尾, 例如: 0.pt, 范围: <0 -

DEF_PATT_TIME_MAX_NUM>

ptData:填入需要传输的数据

73、 读取 MCU flash 中指定的 pattern time 数据

函数原型: int SF_getPTtoAPPFromFlash(const QString & flashName, QByteArray & outPtData, quint8 timeout = DEF_TIMEOUT)

函数说明: 读取 MCU flash 中指定的 pattern time 数据

flashName: MCU flash 中存储的 pattern time 数据文件名, 以.pt 结尾, 例如: 0.pt, 范围: <0 - DEF_PATT_TIME_MAX_NUM>

outPtData:读取过来的 pattern time 数据

74、 从 MCU flash 中获取所有的文件名

函数原型: int SF_getFileNameFromFlash(int & flashUsed, QStringList & fileName)

函数说明: flashUsed: flash 使用率[0-100%]

fileName: pattern time 文件则是以数字开头 .pt 结尾的文件名

pattern bin 文件则是以数字开头 .pb 结尾的文件名

DLP 升级文件则是以 .dlp 结尾的文件名

75、 删除 MCU flash 中指定文件

函数原型: int SF_delFileFromFlash(const QStringList & fileName)

函数说明: 填入相应的文件名, 则可删除对应的文件, 删除完后可读取 SF_getFileNameFromFlash 接口进行刷新界面显示

76、 清除 MCU flash 中所有文件

函数原型: int SF_clrAllFileFromFlash(void)

函数说明: 清除 MCU flash 中所有文件 .pt .pb .dlp 结尾的文件

77、 通过串口升级 DLP 程序

函数原型: int SF_updateDLPProgram(const updateDir_e dir, const QString & fileName, quint8 timeout = DEF_TIMEOUT)

函数说明: 这个 DLP 程序可以直接到 DLP 中, 也可升级到 MCU Flash

```
enum updateDir_e{
    UD_TO_DLP,
    UD_TO_FLASH
}
```

78、 从 MCU flash 中升级 DLP 程序

函数原型: int SF_updateDLPFromFlash(void);

函数说明:

79、 从 DLP 中获取主 DLP flash 的所有程序

函数原型: int SF_getDLPProgramFromDLP(QByteArray & outData, quint8 timeout = DEF_TIMEOUT)

函数说明: 获取主 DLP 中 flash 数据

flashData: 主 DLP 中 flash 数据

80、 MCU 程序升级

函数原型: int SF_updateMCUProgram(const Qstring & fileName, int timeout = DEF_TIMEOUT)

函数说明: 升级 MCU 程序